Defect B — Embedded Curated Indexer Providers (design)

Date: 2026-06-12 **Status:** design (operator-approved approach: "curated Go providers first") **Operator directive:** "On device api app MUST have Jackett embedded same we have the API itself" + "Client app has only 4 providers in onboarding wizard! We do support now many many more via Jackett!" **Relationship to Defect A:** Defect A (fixed in `0deb54e7`) made the client correctly fetch *whatever the api-app serves*. This (Defect B) makes the api-app *serve more than the ~7 natives* — without an external Jackett sidecar — so the on-device experience shows many providers.

`Classification:` project-specific.

1. Problem

The on-device api-app (`digital.vasic.lava.api`) embeds `lava-api-go` as a c-shared `liblavaapi.so`. Its `GET /providers` catalogue is built from `provider.ProviderRegistry.All()`, which on the embedded path (`internal/mobile/mobile.go` `newProductionScraperDeps`) registers only the compiled-in native providers (rutracker, nnmclub, kinozal, archiveorg, gutenberg). The "many more via Jackett" providers require an EXTERNAL Jackett sidecar (`internal/jackett/client.go` → `LAVA_API_JACKETT_URL`), which cannot run inside the on-device embed: Jackett is .NET and cannot be cgo-embedded "same as the API itself".

2. Goal / non-goals

Goal: the on-device api-app's `/providers` exposes a growing, curated set of popular PUBLIC trackers as first-class providers (SEARCH + MAGNET_LINK), each a compiled-in Go `provider.Provider`, with zero external dependency and zero config, covered by falsifiable real-stack tests.

Non-goals (explicitly, anti-bluff honesty):

- NOT a generic Cardigann YAML engine (operator-deferred; that is the "from scratch engine" option not chosen).

- NOT "all 500 Jackett indexers". A curated subset. The §6.J-honest framing: every provider that appears in `/providers` MUST actually work end-to-end; we add them as they are built + tested, never as empty declarations (§6.E).
- The external Jackett sidecar path (`internal/jackett/`) stays UNCHANGED for server deployments that want the full Jackett catalogue.

3. Architecture

Each curated provider is an independent Go package under `internal/provider/curated/<name>/` implementing `provider.Provider` by embedding `provider.BaseProvider` (for the catalogue-metadata defaults) and overriding only what differs. Registration happens in BOTH startup paths that build the registry, exactly where the natives register today:

- `internal/mobile/mobile.go` → `newProductionScraperDeps` (embedded on-device).
- `cmd/lava-api-go/main.go` (standalone server binary).

No router changes: the existing `/providers` handler + the per-provider `/v1/{id}/{op}` routes already iterate the registry, so a registered curated provider is automatically discoverable + searchable.

```
GET /providers                  → registry.All() → [natives…, curated…]   (catalogue)
GET /v1/{curatedId}/search?q=… → registry.Get(id).Search(...)            (results)
```

3.1 Provider unit contract

Each provider package exposes `New() provider.Provider` (and an internal `Client` with an injectable `*http.Client` + base URL so tests substitute a `httptest.Server`). It implements:

Method	Curated behaviour
`ID()`	stable lowercase id, e.g. `"thepiratebay"`, `"yts"`, `"eztv"`
`DisplayName()`	human label, e.g. `"The Pirate Bay"`
`Kind()`	`"native"` (compiled-in; honest — not a remote jackett indexer)
`SupportsAnonymous()`	`true` (no login)
`AuthType()`	`provider.AuthNone`
`Encoding()`	`"UTF-8"`

Method	Curated behaviour
`Capabilities()`	`{CapSearch, CapMagnetLink}` (+ `CapTorrentDownload` only where the tracker serves a real `.torrent`)
`BaseURLs()`	the canonical site URL(s) for informational display
`Search(ctx, opts, cred)`	real HTTP GET to the tracker's JSON/HTML API; parse → `[]SearchItem{Title, InfoHash, MagnetLink, SizeBytes, Seeders, Leechers, …}`
`GetTorrent` / `DownloadFile`	`ErrUnsupported` for magnet-only trackers (§6.E: capability NOT declared)
all other extended methods	`ErrUnsupported`
`Login` / `CheckAuth` / `FetchCaptcha`	anonymous: `Login` no-op success, `CheckAuth` true, `FetchCaptcha` `ErrUnsupported`
`HealthCheck(ctx)`	lightweight GET to the API root/health

Magnet construction: from the BitTorrent v1 `infoHash` + `magnet:?xt=urn:btih:<hash>&dn=<title>&tr=<trackers…>` using the well-known public trackers list (a package constant — protocol data, not a deployment address).

4. Phase plan

Phase 1 — JSON-API, anonymous, no Cloudflare (highest value / lowest risk / real-testable now):

1. `thepiratebay` — `https://apibay.org/q.php?q=<query>&cat=0` → JSON array `[{name, info_hash, seeders, leechers, size, …}]`. Magnet from `info_hash`.
2. `yts` — `https://yts.mx/api/v2/list_movies.json?query_term=<q>` → JSON; each movie's `torrents[]` carry `hash` → magnet.
3. `eztv` — `https://eztvx.to/api/get-torrents?limit=…&keywords=<q>` → JSON `torrents[]` with `magnet_url`, `seeds`, `size_bytes`.

Phase 2 — Cloudflare-gated HTML (reuses the existing FlareSolverr seam): 4. `x1337` (1337x) — HTML search + per-row magnet page; via FlareSolverr when the site is Cloudflare-challenged. 5. `torrentgalaxy` — HTML; FlareSolverr as needed.

Phase 1 ships first as independent, separately-committable providers. Each provider is one commit (impl + tests + registration), so the catalogue grows incrementally and every increment is releasable.

5. Error contract

`Search` maps upstream failures to the package sentinels (`ErrNotFound`/`ErrForbidden`/`ErrNoData`/`ErrUnknown`) — never panics. A provider whose upstream is unreachable returns an error from `Search`; the registry + `/providers` catalogue still lists it (the descriptor is static), so a transiently down tracker does not remove the provider from onboarding (it just returns no results until reachable), matching native behaviour. §6.AC: every error path calls `observability.RecordNonFatal` with `{provider, operation, error_class}`.

6. Testing (Anti-Bluff Pact — per provider, all mandatory)

1. **Fixture parser test** (`<name>_test.go`): a captured REAL upstream response in `testdata/` → drive `Client.Search` against an `httptest.Server` serving the fixture → assert mapped `SearchItem` fields (title, infoHash, magnet contains the hash, sizeBytes, seeders). FALSIFIABILITY: mutate the field-mapping (e.g. read `seeders` from the wrong JSON key) → the assertion fails with a clear message. Recorded in the Bluff-Audit stamp.
2. **Real-network test** (`//go:build realtrackers`): hit the LIVE API with a stable query (e.g. "ubuntu") → assert ⩾1 result with a non-empty 40-hex infoHash + a magnet. Gated so default `go test ./...` makes no outbound calls (mirrors `–PrealTrackers`).
3. **Registry/catalogue test** (`internal/provider` or `internal/handlers`): after registering the curated providers, `GET /providers` body contains the new ids, each with `kind:"native"`, `authType:"NONE"`, `capabilities` ⊇ `["SEARCH","MAGNET_LINK"]`, and `len(providers) > nativeCount`. This is the load-bearing "more than the natives" assertion the operator's bug is about — on user-visible wire state.
4. **Registration parity test:** assert the EMBEDDED path (`mobile.go`) and the STANDALONE path (`main.go`) register the SAME curated set (a regression guard so the on-device app never silently lacks a provider the server has — the §6.J "module-green-but-app-broken" class).

7. Constitutional notes

- **§6.R:** API base URLs follow the EXISTING native pattern — a canonical-domain constant per provider package (the same shape as `rutracker.NewClient("https://rutracker.org/forum")`). These are the provider's own identity (protocol endpoints), not deployment

addresses; consistent with how all 5 natives already encode their base. No ports/IPs/secrets hardcoded.
- **§6.H:** all Phase-1 providers are anonymous — zero credentials. Phase-2 stays anonymous (public trackers); FlareSolverr URL comes from config (`LAVA_API_FLARESOLVERR_URL`).
- **§6.E:** capabilities declared == methods implemented; unsupported → `ErrUnsupported`.
- **§6.Z / §6.AA / §6.Y:** shipping these to the on-device app is a new api-app + embed build → version bump + the §6.Z device-test gate + two-stage distribute apply at distribute time (separate from landing the code).
- **api-source.hash:** every change to `lava-api-go` source drifts the embed hash; `scripts/compute-api-source-hash.sh > core/apiengine/src/main/resources/api-source.hash` runs after each provider lands (the SourceHash contract gate).

8. Honest scope statement (for CHANGELOG / release notes)

"Adds built-in public-tracker providers (The Pirate Bay, YTS, EZTV, …) to the on-device API — searchable with magnet links, no external Jackett required. This is a curated, growing set, not a full Jackett mirror; server deployments can still point at an external Jackett for the complete indexer catalogue."

Post-1076 Revision (2026-06-26)

1076 Distribute Outcome

Lava-Android-1.3.12-1076 + api-app 0.2.11-22 were distributed and reported to testers. The operator's post-distribute manual testing concluded "practically almost nothing has been fixed". Root cause: the §6.Z device gate executed **only Challenge00CrashSurvivalTest (C00)** while the cycle's CHANGELOG claimed fixes to search, provider-selection, onboarding, and display. The curated-providers code path — which this spec governs — was never exercised on-device during the gate. This is now a constitutionally prohibited pattern per **§6.AK** (Cycle-Coverage Device Gate, added 2026-06-26): a `Challenge00CrashSurvivalTest`-only gate NEVER satisfies the distribute requirement when the cycle claims provider/UI fixes.

Curated Providers That Actually Shipped

The on-device api-app now exposes **7 curated providers** (in addition to the 5 natives). These are the providers that shipped at 1076:

1. **The Pirate Bay** (`thepiratebay`) — JSON API via `https://apibay.org/q.php?q=<query>&cat=0`. Phase 1, anonymous, no Cloudflare.
2. **YTS** (`yts`) — JSON API via `https://yts.mx/api/v2/list_movies.json?query_term=<q>`. Phase 1.
3. **Torrents-CSV** (`torrentscsv`) — added after the spec was written; supplements the Phase 1 set.
4. **Knaben** (`knaben`) — added after the spec was written.
5. **Nyaa** (`nyaa`) — added after the spec was written.
6. **BitSearch** (`bitsearch`) — added after the spec was written.
7. **TorrentDownloads** (`torrentdownloads`) — added after the spec was written.

The original Phase 1 set (TPB, YTS, EZTV) was extended during implementation. **EZTV** was resolved to **SKIP**: the operator determined there is no honest anonymous-search path for EZTV on the on-device embed (the upstream requires either a Cloudflare-bypass seam or a registered account for consistent API access), so it was removed from the curated list rather than shipped with a broken search flow.

Phase 2 Cloudflare Status

Phase 2 of the spec proposed **1337x** (`x1337`) as a Cloudflare-gated provider reusing the existing FlareSolverr seam. As of 1076, 1337x remains **blocked** — its upstream Cloudflare challenge is active and the FlareSolverr seam (`LAVA_API_FLARESOLVERR_URL`) exists in the codebase but is not yet registered as a routing path for the curated `x1337` provider. A dedicated FlareSolverr integration step is needed before 1337x can ship. The upstream `torrentgalaxy` Phase 2 candidate has not been implemented; it remains deferred.

Crashlytics Wiring

The api-app Crashlytics non-fatal telemetry for curated providers (described in sections 5 and 7 as required per §6.AC) was completed in commit `07f83eef`. Every error path in each curated provider now records a non-fatal event with `{provider, operation, error_class}` context, landing in the same Firebase Crashlytics dashboard as the client-side events. This means operator visibility into upstream-failure patterns for curated providers is active.

§6.AK Impact on Testing Strategy

Section 6's testing strategy must be updated to reflect the new §6.AK gate requirements. Every curated provider that ships in a distribute cycle now demands an **EXECUTED + PASSED** covering device Challenge before that version can be distributed:

- The registry/catalogue test (section 6 item 3) must run on-device (not just JVM) as part of the §6.Z evidence file.
- The registration parity test (section 6 item 4) must be linked to an attestation row captured from the actual api-app binary about to be distributed — compilation green is never sufficient.
- Any new curated provider added after 1076 must have its falsifiability rehearsal recorded per §6.AK clause 2: the test fails RED against a deliberately broken provider (e.g., search returns empty, or the provider is absent from the registry) before the GREEN run against the fix. The RED run evidence must appear in the commit body (Bluff-Audit stamp).
- The `Challenge00CrashSurvivalTest` alone is NEVER sufficient (§6.AK clause 1).

Known Issues Affecting Spec Assumptions

- **LVA-008:** The `C11` ConnectedAndroidTest navigation-teardown crash blocks nested-route device Challenges. Curated-provider full-flow tests (navigate to search, select a curated provider from the dynamically-populated list, enter query, observe results) may produce false-negative test infrastructure failures until this is resolved.
- **Search engine timeout mismatch:** The engine's 18s deadline vs client 30s read-timeout produces a window where the curated provider's upstream response arrives between 18-30s and is silently discarded. Curated providers with slower upstreams (e.g. The Pirate Bay's `apibay.org` under load) are more likely to hit this.
- **AuthInterceptor handoff-key overwrite (H1):** Fixed in 1072. Curated providers are anonymous (no auth), but the per-request routing middleware still sends authentication headers; a middleware-state corruption could affect request routing for curated providers as well.
- **Partial-failure Error→Empty display fix:** Landed in `1cbf364c`. Before this fix, if one curated provider returned an error and another succeeded, the screen rendered empty instead of showing the successful results. This directly affects the user-visible experience of the curated-provider search flow.

Updated Change History

Date	Author	Change
2026-06-12	Agent	Initial spec
2026-06-26	Agent	Post-1076 revision: documented C00-only gate outcome, listed the 7 shipped curated providers, documented EZTV→SKIP, Cloudflare status for 1337x, Crashlytics wiring in `07f83eef`, §6.AK testing gate, known issues, and this change history entry